

CREDIXA: LIVE CLOUD EVALUATION & TESTING PROCEDURE

Comprehensive Step-by-Step Guide for Test Scenarios 1 through 10

Evaluation Context: This document provides the official testing procedure for evaluating **Credixa** via live cloud deployment links using the pre-generated test documents in the `testing_docs/test_scenarios/` repository. This guide covers the complete suite of **Scenarios 1 through 10**, evaluating seamless happy-path underwriting, high Debt-to-Income (DTI) rejections, fraud detection, advanced OCR discrepancy handling, session recovery, identity cross-verification, automated scholarship scaling, behavioral risk detection, SLA escalation, and returning user fast-tracking.

1. Live Cloud Ecosystem & Endpoints

All testing should be conducted directly on the live production environment. No local database or server installation is required.

Service Pillar	Live Cloud Endpoint	Role & Description
Web Portal (React)	https://credixa-8w1w.onrender.com/	Student Onboarding, Co-Applicant Linking, & Admin Verification Portal.
API Gateway	https://credixa-backend-7r4k.onrender.com/	Node.js/Express REST API Gateway, JWT Auth, and Redis Rate Limiter.
AI Risk Engine	https://credixa-risk-engine.onrender.com/docs	Python FastAPI Swagger UI for XGBoost Underwriting & HuggingFace OCR.
Database Vault	Managed Cloud PostgreSQL 16	Relational data vault and immutable audit log storage.

2. General Evaluation Instructions & Document Access

For each scenario below, navigate to the local repository directory `testing_docs/test_scenarios/` to locate the pre-generated PDF and JPEG files required for upload. Ensure you test using an incognito browser window or clear session cookies when switching between Student and Institution Admin personas.

Scenario 1: Rahul Sharma – Happy Path: Seamless Approval & Instant Disbursement

Test Folder: [testing_docs/test_scenarios/01_Rahul_Sharma/](#)

Objective: Evaluate the complete happy path lifecycle for a high-merit student with clean financial records, from registration to instant AI underwriting and institutional disbursement.

Step-by-Step Execution Procedure:

- 1. Register Student:** Go to <https://credixa-8wlv.onrender.com/register/student> and create an account for **Rahul Sharma**. Select an active college (e.g., Apex Institute of Technology).
- 2. KYC & Co-Applicant:** Complete Step 1 (KYC) using PAN APZPS2167S and Aadhaar 000011223344. In Step 2, link Mother Priya Sharma as Co-Applicant.
- 3. Document Vault Upload:** From [01_Rahul_Sharma/](#), upload academic marksheets (10th_marksheet.pdf, 12th_marksheet.pdf), admission letter, fee structure, and bank statements (rahul_bank_statement.pdf, rahul's_mother_bank_statement.pdf).
- 4. AI Underwriting & OCR:** Enable OCR Digitization & AI Underwriting, then click **Run Analysis & Submit**. The system parses tabular marksheets via HuggingFace LLM and evaluates bank transaction velocity via XGBoost.
- 5. Loan Application:** On the Student Dashboard ([/student-dashboard](#)), observe the high Omniscore (>800). Request a BNPL loan for **₹35,000** over 12 months.
- 6. Institution Verification:** Login to the Admin Dashboard ([/admin-dashboard](#)) as an institution admin. Review Rahul's OCR verified standing and click **Approve Enrollment** → Loan state transitions to *Disbursed*.

Expected System Outcome: High Omniscore (>800), full BNPL credit limit unlocked, and instant disbursement upon institutional enrollment verification.

Scenario 2: Kamran Khan – Unhappy Path: High Debt-to-Income (DTI) Rejection

Test Folder: [testing_docs/test_scenarios/02_Kamran_Khan/](#)

Objective: Evaluate the XGBoost risk classifier's ability to identify severe liquidity stress and excessive debt burdens, triggering an automated loan rejection.

Step-by-Step Execution Procedure:

- 1. Register Student:** Create an account for student **Kamran Khan** at [/register/student](#). Complete Step 1 (KYC) and link Father Tariq Khan as Co-Applicant in Step 2.
- 2. Upload Financial Documents:** In Step 3, upload `kamran's_father_bank_statement.pdf` from `02_Kamran_Khan/` along with academic marksheets.
- 3. Examine Financial Stress:** Note that the father's bank statement demonstrates severe liquidity stress, showing **₹41,000** in existing monthly EMI deductions against **₹45,000** retail sales receipts (Debt-to-Income ratio > 91%) and a closing balance of only **₹500**.
- 4. Execute AI Underwriting:** Click **Run Analysis & Submit** to feed the transaction data into the XGBoost classifier.
- 5. Verify Automated Rejection:** Observe that the AI Risk Engine flags high credit risk and assigns an Omniscore < 500.
- 6. Check Loan Status:** Verify that any loan request on the dashboard is restricted or automatically placed in *Rejected / High Risk* status to protect against default.

Expected System Outcome: Automated rejection with an Omniscore < 500 due to a Debt-to-Income (DTI) ratio exceeding 91% and severe liquidity stress.

Scenario 3: Harpreet Singh – Unhappy Path: Fraud Detection & Document Tampering

Test Folder: [testing_docs/test_scenarios/03_Harpreet_Singh/](#)

Objective: Test the AI Risk Engine's fraud detection algorithms against forged calendar dates and mathematical balance inconsistencies in uploaded financial documents.

Step-by-Step Execution Procedure:

- 1. Register Student:** Create an account for student **Harpreet Singh** at [/register/student](#). Complete Step 1 (KYC) and link Father Daljit Singh as Co-Applicant.
- 2. Upload Tampered Statement:** In Step 3, upload harpreet's_father_bank_statement_TAMPERED.pdf from 03_Harpreet_Singh/.
- 3. Examine Forgeries:** Note that the uploaded PDF contains intentional mathematical impossibilities (Page 1 closing balance is ■4,50,000 while Page 2 starting balance brought forward is ■14,50,000 without a credit entry) and forged calendar dates (30-Feb-2026 and 31-Apr-2026).
- 4. Trigger AI Analysis:** Click **Run Analysis & Submit** to initiate document validation.
- 5. Observe Fraud Flag:** Notice that the AI Risk Engine catches the mathematical impossibility and forged dates during transaction parsing.
- 6. Verify Security Lock:** Verify that the system instantly flags suspected fraud, locks the application from further progression, and queues the profile for manual investigation.

Expected System Outcome: Immediate fraud detection and application lock triggered by mathematical balance inconsistencies and forged calendar dates.

Scenario 4: Rhea D'Souza – Discrepancy Resolution Loop via Document Re-upload

Test Folder: [testing_docs/test_scenarios/04_Rhea_DSouza/](#)

Objective: Evaluate system behavior when an uploaded document fails OCR readability checks, triggering an admin rejection and subsequent student re-upload recovery loop.

Step-by-Step Execution Procedure:

- 1. Register / Login:** Go to <https://credixa-8wlv.onrender.com/register/student> and create an account for **Rhea D'Souza** (or login if registered). Select an active college.
- 2. KYC & Co-Applicant:** Complete KYC identity verification and link Father Michael D'Souza as Co-Applicant.
- 3. Upload Blurry Document:** In Step 3 (Document Upload), upload the intentionally degraded file `rhea_fee_structure_REJECTED.jpeg` along with valid marksheets and bank statements.
- 4. Submit for Underwriting:** Click **Run Analysis & Submit**. Notice the application enters *Under Review* status.
- 5. Admin Rejection:** Login to the Institution Admin portal ([/admin-dashboard](#)). Locate Rhea's application and click **Reject / Flag Discrepancy** on the fee structure due to unreadable OCR text.
- 6. Student Re-upload:** Switch back to Rhea's Student Dashboard. Observe the discrepancy alert requesting document re-upload. Upload the pristine PDF: `rhea_fee_structure_APPROVED.pdf`.
- 7. Final Approval:** Admin re-verifies the newly uploaded clear PDF and clicks **Approve Enrollment**. The loan state successfully transitions to *Approved / Disbursed*.

Expected System Outcome: The system correctly flags unreadable OCR text, pauses loan progression, alerts the student, and seamlessly resumes underwriting upon receiving a valid document re-upload.

Scenario 5: Rohan Mehta – Parent Drops Off & Session Abandonment Recovery

Test Folder: [testing_docs/test_scenarios/05_Rohan_Mehta/](#)

Objective: Test session state persistence and data integrity when a user abandons the onboarding flow midway and returns later to complete the application.

Step-by-Step Execution Procedure:

- 1. Student Registration:** Create an account for student **Rohan Mehta** at [/register/student](#). Complete Step 1 (KYC Verification).
- 2. Co-Applicant Linking:** In Step 2, input details for Father Suresh Mehta and save the co-applicant profile.
- 3. Simulate Session Abandonment:** Abruptly close the browser tab or click **Logout** before uploading any academic or financial documents.
- 4. Session Recovery:** Re-open the browser after a few moments and login as Rohan Mehta at [/login](#).
- 5. Verify Progress Persistence:** Observe that Credixa automatically restores the exact onboarding state (Step 3: Document Upload) without losing KYC or Co-Applicant data.
- 6. Complete Application:** Upload rohan_fee_structure_happy.pdf and bank statements, then click **Run Analysis & Submit**.
- 7. Dashboard Verification:** Verify that the AI Risk Engine computes the Omniscore and unlocks the BNPL loan request module.

Expected System Outcome: Credixa preserves 100% of previously entered KYC and Co-Applicant data across browser sessions, enabling frictionless resume-from-drop-off functionality.

Scenario 6: Pooja Nair – Identity Mismatch & OCR Validation Failure

Test Folder: [testing_docs/test_scenarios/06_Pooja_Nair/](#)

Objective: Evaluate the OCR cross-verification engine's ability to detect inconsistencies between uploaded KYC identity documents and bank statement account holder names.

Step-by-Step Execution Procedure:

1. **Register Student:** Create an account for student **Pooja Nair**. Complete Step 1 (KYC) and link Father K. R. Nair as Co-Applicant in Step 2.
2. **Upload Mismatched Documents:** In Step 3 (Document Upload), navigate to 06_Pooja_Nair/ and upload:
 3. • **Co-Applicant KYC:** test_coapplicant_father_kyc_raw.pdf (where the identity name reads 'Amit Patel').
 4. • **Bank Statement:** pooja's_father_bank_statement_MISMATCH.pdf (where account holder name reads 'K. R. Nair' or 'Nair & Sons Enterprises').
5. **Trigger AI Analysis:** Click **Run Analysis & Submit** to initiate the OCR cross-verification pipeline.
6. **Observe Cross-Verification Failure:** Notice that the system catches the identity mismatch between the government KYC document and the bank statement account holder name.
7. **Verify Progression Block:** Verify that the system flags an OCR Identity Mismatch error and blocks loan sanctioning until consistent identity documents are provided.

Expected System Outcome: The AI Risk Engine flags an OCR Identity Mismatch between the government ID and bank statement, blocking loan approval to prevent identity fraud.

Scenario 7: Arjun Deshmukh – Partial Scholarship & Over-Borrowing Adjustment

Test Folder: [testing_docs/test_scenarios/07_Arjun_Deshmukh/](#)

Objective: Test the OCR parser's ability to identify scholarship deduction line items on university fee structures and dynamically adjust loan credit limits to prevent over-borrowing.

Step-by-Step Execution Procedure:

- 1. Register Student:** Create an account for student **Arjun Deshmukh** (BITS Pilani applicant). Complete KYC and Co-Applicant linking.
- 2. Upload Scholarship Fee Structure:** In Step 3, upload `arjun_fee_structure_scholarship.pdf` along with marksheets and bank statements.
- 3. Examine Document Content:** Note that the fee structure lists Base Tuition as **₹10,00,000**, but contains an official line item for **-₹5,00,000** BITS Merit Scholarship, making the Net Payable amount **₹5,00,000**.
- 4. Run AI Underwriting:** Click **Run Analysis & Submit**. The HuggingFace LLM extractor parses the itemized fee breakdown.
- 5. Attempt Over-Borrowing:** On the Student Dashboard, attempt to request a BNPL loan amount of **₹8,00,000** (exceeding the net payable fee).
- 6. Verify Automated Limit Scaling:** Observe that Credixa's underwriting logic automatically caps and scales down the maximum borrowable limit to exactly **₹5,00,000** to match the OCR-verified net tuition obligation.

Expected System Outcome: The system successfully identifies the 50% merit scholarship via OCR and automatically restricts the BNPL loan sanction limit from **₹10,00,000** to the net payable **₹5,00,000**.

Scenario 8: Darius Mistry – High-Risk Behaviour Pattern & Gambling Flags

Test Folder: [testing_docs/test_scenarios/08_Darius_Mistry/](#)

Objective: Evaluate the XGBoost machine learning classifier's capability to detect high-risk behavioral transaction patterns (online gambling/betting) and penalize creditworthiness.

Step-by-Step Execution Procedure:

- 1. Register Student:** Create an account for student **Darius Mistry**. Complete KYC and Co-Applicant linking (Father Farokh Mistry).
- 2. Upload Gambling Statement:** In Step 3, upload `darius's_father_bank_statement_GAMBLING.pdf` as the Co-Applicant financial document.
- 3. Examine Transaction Patterns:** Note that while the father earns a high salary (₹2,50,000/month), the statement contains frequent high-value debits to betting and fantasy gaming merchants (Dream11, Betway, Parimatch Casino, RummyCircle) accounting for >35% of monthly income.
- 4. Execute Underwriting:** Click **Run Analysis & Submit** to feed the transaction logs into the XGBoost classifier.
- 5. Verify Score Penalty:** On the Student Dashboard (or Admin Dashboard), observe that the AI Risk Engine applies a severe credit score penalty due to high-risk behavioral patterns, dropping the Omniscore below 550 and triggering an automatic rejection or compliance review.

Expected System Outcome: The XGBoost classifier detects >35% monthly income outflow to gambling merchants, applying a drastic score reduction (Omniscore < 550) and flagging high behavioral credit risk.

Scenario 9: Divya Iyer – Institution Delays Verification & SLA Escalation

Test Folder: [testing_docs/test_scenarios/09_Divya_Iyer/](#)

Objective: Test system monitoring of institutional verification queues and automated SLA escalation alerts when loan approvals are delayed by university administrators.

Step-by-Step Execution Procedure:

- 1. Register Student:** Create an account for student **Divya Iyer**. Complete KYC, Co-Applicant linking, and upload clean documents (divya_fee_structure_happy.pdf, divya's_father_bank_statement.pdf).
- 2. Run AI Analysis:** Submit documents for underwriting. The AI Risk Engine awards an excellent Omniscore (950/1000) and places the loan in *Under Review* status awaiting university verification.
- 3. Simulate Verification Delay:** In a testing environment, simulate the application sitting in the institution's verification queue beyond the standard SLA window (e.g., > 7 days without action).
- 4. Monitor Admin SLA Alert:** Login to the Institution Admin portal ([/admin-dashboard](#)). Observe the SLA escalation warning badge and priority indicator highlighted next to Divya's application.
- 5. Execute Verification:** Admin reviews the escalated application and clicks **Approve Enrollment**. Verify that the loan immediately transitions to *Disbursed*.

Expected System Outcome: Credixa tracks institutional verification turnaround times, triggers automated SLA escalation alerts for pending approvals, and ensures rapid disbursement upon verification.

Scenario 10: Abbas Ali – Multi-Semester Fast Track for Returning Users

Test Folder: [testing_docs/test_scenarios/10_Abbas_Ali/](#)

Objective: Evaluate returning user experience, baseline data reuse, and fast-track automated underwriting for students seeking financing for subsequent academic semesters.

Step-by-Step Execution Procedure:

- 1. Register / Access Returning User:** Login or create account for returning student **Abbas Ali** (IIIT Hyderabad applicant).
- 2. Context & Baseline:** Assume Abbas previously completed Semester I BNPL financing cleanly with zero repayment defaults, seeding his verified KYC identity and financial baseline in the database.
- 3. Initiate Semester II Loan:** On the Student Dashboard, click **Request New BNPL Financing** for Semester II.
- 4. Observe Fast-Track Bypass:** Notice that Credixa automatically bypasses redundant KYC identity checks and 6-month bank statement re-uploads by referencing verified baseline standing.
- 5. Upload New Fee Breakdown:** Upload only the updated document: abbas_fee_structure_semester2.pdf (Total Payable: ₹3,75,000).
- 6. Instant Sanction:** Click **Apply for BNPL Financing**. Observe that the AI Risk Engine validates the new fee structure against existing credit standing and approves the loan in **< 30 seconds**.

Expected System Outcome: The system reuses established KYC and financial baseline data, bypassing redundant document uploads and achieving fast-track loan sanctioning in under 30 seconds.

3. Direct API & Swagger UI Verification (Technical Grading)

For technical grading of backend REST APIs, JWT authentication, and machine learning inference models without frontend UI interaction, use the live Swagger UI and Health endpoints below:

Endpoint URL	HTTP Method	Technical Purpose & Verification Method
/analyze/bank-statement (Risk Engine Swagger)	POST	Upload raw PDF bank statement. Returns JSON containing parsed transaction velocity, savings rate, DTI ratio, and XGBoost risk classification.
/analyze/ocr-document (Risk Engine Swagger)	POST	Upload academic transcript or fee structure. Returns HuggingFace LLM digitized JSON containing student name, roll number, CGPA, and total fees.
/score/omniscore (Risk Engine Swagger)	POST	Input feature vector (academic score, DTI, savings rate). Returns computed 300-1000 Omniscore and risk tier categorization.
/health (API Gateway)	GET	Returns Node.js gateway health, PostgreSQL database connectivity status, and Redis cache latency.

Credixa Evaluation Document: Generated automatically from repository testing specifications. For additional assistance or technical inquiries during grading, please refer to the master repository at <https://github.com/mdkamranalam/credixa>.